Laporan Penerapan Modul Pertemuan 9

Aplikasi TanyaBareng

Tujuan dan ruang lingkup

Pengembangan ini menerapkan komponen antarmuka dari modul pertemuan 9 pada aplikasi TanyaBareng. Perubahan meliputi ChipGroup yang dapat dipilih, background header profil berbentuk wave, susunan lencana menggunakan GridLayout, serta Tutorial Screen yang terdiri dari tiga halaman.

Layout tutorial menggunakan ConstraintLayout. GridLayout digunakan pada bagian lencana profil. ChipGroup dibatasi pada perubahan status pilihan; pemilihan chip belum mengubah daftar pertanyaan. Snackbar, WebView, dan latihan mandiri modul tidak termasuk pembahasan laporan ini.

Struktur program

Package Tutorial berada sejajar dengan Home, Cari, Notifikasi, dan Profil. Di dalamnya terdapat TutorialActivity, TutorialFragmentAdapter, Tutorial1Fragment, Tutorial2Fragment, dan Tutorial3Fragment. Ketiga Fragment menampilkan layout XML masing-masing melalui onCreateView().

Library yang digunakan

build.gradle.kts

```
41    implementation(libs.androidx.gridlayout)
42    implementation(libs.androidx.viewpager2)
43    implementation(libs.dotsindicator)
```

Gambar 1. Dependency GridLayout, ViewPager2, dan DotsIndicator pada app/build.gradle.kts.

Versi library diatur melalui gradle/libs.versions.toml: GridLayout 1.0.0, ViewPager2 1.1.0, dan DotsIndicator 5.1.0. Penggunaan version catalog membuat pemanggilan dependency pada modul app lebih ringkas.

Acuan implementasi

Acuan materi adalah Modul Praktikum Pemrograman Mobile Pertemuan 9. Struktur Fragment, adapter, daftar fragment di Activity, serta pemasangan DotsIndicator mengikuti contoh modul. Isi tutorial dan tombol navigasi menyesuaikan desain aplikasi TanyaBareng.

1 ChipGroup yang dapat dipilih

ChipGroup mengelompokkan pilihan mata pelajaran pada Beranda. singleSelection membatasi pilihan menjadi satu chip, sedangkan selectionRequired mempertahankan setidaknya satu pilihan. Chip Semua diberi checked bernilai true sebagai pilihan awal.

fragment_home.xml

```
147   <com.google.android.material.chip.ChipGroup
148       android:id="@+id/chipGroupSubject"
149       android:layout_width="wrap_content"
150       android:layout_height="wrap_content"
151       app:singleSelection="true"
152       app:selectionRequired="true"
153       app:chipSpacingHorizontal="8dp"
154       app:singleLine="true">
155
156       <com.google.android.material.chip.Chip
157           android:id="@+id/chipHomeSubjectAll"
158           style="@style/TanyaBareng.Chip"
159           android:layout_width="wrap_content"
160           android:layout_height="wrap_content"
161           android:checked="true"
162           android:text="@string/subject_all" />
```

Gambar 2. Potongan ChipGroup dan chip Semua pada fragment_home.xml.

Setiap chip memiliki ID yang berbeda agar status pilihannya dapat dikelola ChipGroup. Pola pemilihan juga diterapkan pada halaman Cari dan form Buat Pertanyaan. Interaksi ini baru menandai pilihan; logika filter data belum ditambahkan.

Hasil pilihan ChipGroup

[Tempel screenshot hasil aplikasi di sini]

Tampilkan chip Matematika atau pilihan lain dalam keadaan terpilih.

2 Warna dan indikator pilihan chip

Style TanyaBareng.Chip membuat chip dapat dipilih melalui checkable. Warna background, teks, dan garis tepi menggunakan color state list. Ikon centang menjadi penanda tambahan ketika chip dipilih.

themes.xml

```
259   <style name="TanyaBareng.Chip" parent="Widget.Material3.Chip.Filter">
260       <item name="android:checkable">true</item>
261       <item name="android:textColor">@color/chip_selection_text</item>
262       <item name="chipBackgroundColor">@color/chip_selection_bg</item>
263       <item name="chipStrokeColor">@color/chip_selection_stroke</item>
264       <item name="checkedIcon">@drawable/ic_check</item>
265       <item name="checkedIconVisible">true</item>
266       <item name="checkedIconTint">@color/chip_selection_text</item>
267       <item name="chipStrokeWidth">1dp</item>
268       <item name="chipMinHeight">36dp</item>
269       <item name="chipCornerRadius">999dp</item>
270       <item name="chipIconSize">16dp</item>
271       <item name="android:textSize">13sp</item>
272   </style>
```

Gambar 3. Style chip pada themes.xml.

chip_selection_bg.xml

```
1     <?xml version="1.0" encoding="utf-8"?>
2     <selector xmlns:android="http://schemas.android.com/apk/res/android">
3         <item android:state_checked="true" android:color="@color/primary" />
4         <item android:color="@color/surface" />
5     </selector>
```

Gambar 4. Perubahan warna background berdasarkan state_checked.

Selector memeriksa state_checked terlebih dahulu. Saat terpilih, background memakai warna primary; keadaan lainnya memakai surface. File chip_selection_text.xml mengatur warna teks, sedangkan chip_selection_stroke.xml mengatur warna garis tepi. Ketiganya berada pada direktori res/color.

Chip mata pelajaran seperti Matematika tetap memakai warna pastel sesuai desain. Status terpilihnya ditunjukkan melalui ikon centang dan garis tepi, sehingga tidak semua chip berubah menjadi background hijau.

3 Background header profil berbentuk wave

Header profil menggunakan vector drawable bg_header_wave.xml. Bentuk gelombang dibuat melalui pathData, kemudian dipakai sebagai background pada viewHeaderBg. Komponen profil lainnya tetap mengikuti susunan yang sudah ada.

bg_header_wave.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="360dp"
    android:height="300dp"
    android:viewportWidth="360"
    android:viewportHeight="300">

    <!-- Bagian bawah transparan memperlihatkan background halaman di balik kartu. -->
    <path
        android:fillColor="@color/primary"
        android:pathData="M0,0 L360,0 L360,252 C270,279 90,225 0,258 Z" />
</vector>
```

Gambar 5. Vector drawable pembentuk gelombang pada bg_header_wave.xml.

Perintah C pada pathData membentuk kurva kubik di bagian bawah header. Area di luar path tetap transparan sehingga background halaman terlihat di bawah gelombang. Warna isi path mengikuti warna primary aplikasi.

fragment_profil.xml

```xml
<View
    android:id="@+id/viewHeaderBg"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:background="@drawable/bg_header_wave"
    app:layout_constraintBottom_toBottomOf="@id/spaceOverlap"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Gambar 6. Pemasangan background wave pada fragment_profil.xml.

Hasil header profil

[Tempel screenshot hasil aplikasi di sini]

Tampilkan bagian atas Profil sampai lengkungan di atas kartu statistik.

4 GridLayout pada bagian lencana

Bagian Lencana pada halaman Profil menggunakan GridLayout dengan tiga kolom dan satu baris. Tiga MaterialCardView ditempatkan pada kolom 0, 1, dan 2. Setiap kartu menggunakan layout_columnWeight bernilai 1 agar memperoleh bagian lebar yang seimbang.

fragment_profil.xml

```
326   <androidx.gridlayout.widget.GridLayout
327       android:id="@+id/gridBadges"
328       android:layout_width="0dp"
329       android:layout_height="wrap_content"
330       android:layout_marginHorizontal="16dp"
331       android:layout_marginTop="10dp"
332       app:columnCount="3"
333       app:rowCount="1"
334       app:useDefaultMargins="false"
335       app:layout_constraintEnd_toEndOf="parent"
336       app:layout_constraintStart_toStartOf="parent"
337       app:layout_constraintTop_toBottomOf="@id/tvBadgesTitle">
338
339       <com.google.android.material.card.MaterialCardView
340           android:id="@+id/badge1"
341           style="@style/Widget.Material3.CardView.Outlined"
342           android:layout_width="0dp"
343           android:layout_height="wrap_content"
344           app:layout_row="0"
345           app:layout_column="0"
346           app:layout_columnWeight="1"
347           app:layout_gravity="fill"
```

Gambar 7. Konfigurasi GridLayout dan posisi kartu lencana pertama.

Lebar kartu diatur 0dp, kemudian dibagikan melalui bobot kolom. layout_gravity bernilai fill membuat kartu mengisi ruang sel. Jarak antarkartu diatur melalui margin, sedangkan isi setiap kartu tetap menggunakan ConstraintLayout.

Grid ini menyusun lencana Penjawab Pertama, 5× Terbaik, dan Rajin 7 Hari. Tab Pertanyaan Saya dan Jawaban Saya ditempatkan di bawah gridBadges agar tidak bertumpuk dengan kartu lencana.

Hasil susunan lencana

[Tempel screenshot hasil aplikasi di sini]

Tampilkan ketiga kartu lencana dalam satu baris pada halaman Profil.

5 Fragment dan adapter Tutorial Screen

Tutorial Screen terdiri dari tiga Fragment. Masing-masing Fragment mewarisi Fragment() dan mengembalikan layout melalui onCreateView(). Contoh berikut menunjukkan halaman pertama; halaman kedua dan ketiga memakai pola yang sama dengan layout yang berbeda.

Tutorial1Fragment.kt

```
10    class Tutorial1Fragment : Fragment() {
11        override fun onCreateView(
12            inflater: LayoutInflater, container: ViewGroup?,
13            savedInstanceState: Bundle?
14        ): View? {
15            return inflater.inflate(R.layout.fragment_tutorial1, container, false)
16        }
17    }
```

Gambar 8. Tutorial1Fragment menampilkan fragment_tutorial1.xml.

TutorialFragmentAdapter menerima Activity dan List<Fragment>. getItemCount() mengembalikan jumlah halaman, sedangkan createFragment(position) mengambil Fragment pada posisi yang diminta ViewPager2.

TutorialFragmentAdapter.kt

```
7     class TutorialFragmentAdapter(
8         activity: FragmentActivity,
9         private val fragments: List<Fragment>
10    ) : FragmentStateAdapter(activity) {
11        override fun getItemCount(): Int = fragments.size
12        override fun createFragment(position: Int): Fragment = fragments[position]
13    }
```

Gambar 9. TutorialFragmentAdapter mengikuti pola daftar fragment pada modul.

Ketiga halaman memperkenalkan cara mengajukan pertanyaan, menjawab dan mendukung teman, serta memilih jawaban terbaik. Ilustrasinya disusun dari komponen XML seperti TextView, MaterialCardView, dan ConstraintLayout.

6 Menghubungkan halaman dan indikator

Daftar halaman dibuat di TutorialActivity dengan listOf(). Urutan objek dalam daftar menentukan urutan tutorial. Daftar tersebut diberikan kepada adapter, lalu adapter dipasang ke ViewPager2. DotsIndicator dihubungkan setelah adapter tersedia.

TutorialActivity.kt

```
34    val fragmentsList = listOf(
35        Tutorial1Fragment(),
36        Tutorial2Fragment(),
37        Tutorial3Fragment()
38    )
39    val adapter = TutorialFragmentAdapter(this, fragmentsList)
40    binding.tutorialPager.adapter = adapter
41    binding.dotIndicator.attachTo(binding.tutorialPager)
```

Gambar 10. Daftar fragment, adapter, dan DotsIndicator pada TutorialActivity.

Tutorial dibuka melalui item action_tutorial pada toolbar Beranda. Intent mengarahkan pengguna ke TutorialActivity. Activity ini juga didaftarkan pada AndroidManifest.xml dengan nama .Tutorial.TutorialActivity.

HomeFragment.kt

```
78    R.id.action_tutorial -> {
79        startActivity(Intent(requireContext(), TutorialActivity::class.java))
80        true
81    }
```

Gambar 11. Aksi menu Beranda untuk membuka TutorialActivity.

MenuProvider terikat pada viewLifecycleOwner dengan Lifecycle.State.RESUMED. Dengan pengaturan ini, menu Beranda mengikuti masa aktif tampilan HomeFragment. Lokasi package Tutorial sudah dipisahkan dari Home sehingga kedua package berada pada tingkat yang sama.

Alur halaman

Halaman pertama menjelaskan cara bertanya. Halaman kedua menjelaskan jawaban dan dukungan. Halaman ketiga menjelaskan jawaban terbaik dan poin. Pergeseran halaman dapat dilakukan dengan swipe atau tombol Lanjut; titik indikator mengikuti halaman yang aktif.

7 Tombol Lewati Lanjut dan Mulai

Tombol Lanjut memindahkan ViewPager2 ke halaman berikutnya. Pada halaman terakhir, tombol berubah menjadi Mulai dan menjalankan finish() untuk menutup tutorial. Tombol Lewati juga menutup tutorial dan hanya ditampilkan pada dua halaman pertama.

TutorialActivity.kt

```
46    binding.btnSkip.setOnClickListener { finish() }
47    binding.btnNext.setOnClickListener {
48        val page = binding.tutorialPager.currentItem
49        if (page == fragmentsList.lastIndex) {
50            finish()
51        } else {
52            binding.tutorialPager.setCurrentItem(page + 1, true)
53        }
54    }
```

Gambar 12. Penanganan tombol Lewati dan Lanjut atau Mulai.

TutorialActivity.kt

```
57    private fun updateNavigation(position: Int) {
58        val pageCount = binding.tutorialPager.adapter?.itemCount ?: return
59        val isLastPage = position == pageCount - 1
60        binding.btnSkip.visibility = if (isLastPage) View.INVISIBLE else View.VISIBLE
61        binding.btnNext.setText(if (isLastPage) R.string.tutorial_start else
          R.string.tutorial_next)
62    }
```

Gambar 13. Penyesuaian tombol berdasarkan halaman yang aktif.

OnPageChangeCallback memanggil updateNavigation() ketika halaman berpindah, termasuk saat pengguna melakukan swipe. Callback dilepas pada onDestroy(). Tombol navigasi ini ditambahkan untuk mengikuti desain tutorial TanyaBareng.

Penyimpanan posisi halaman secara manual dan penanganan tombol Back khusus tidak digunakan. Tombol Back mengikuti perilaku bawaan Activity. Ukuran header ilustrasi diatur pada XML dengan minHeight 400dp dan wrap_content agar dapat mengikuti tinggi isi kartu.

8 Hasil Tutorial Screen

Dokumentasi hasil menampilkan ketiga halaman tutorial secara berurutan. Pada halaman pertama dan kedua, tombol utama bertuliskan Lanjut. Pada halaman ketiga, tombol utama menjadi Mulai dan tombol Lewati disembunyikan.

Tutorial halaman pertama

[Tempel screenshot hasil aplikasi di sini]

Judul Tanya apa saja kapan saja, kartu pertanyaan, indikator pertama, dan tombol Lanjut.

Tutorial halaman kedua

[Tempel screenshot hasil aplikasi di sini]

Judul Jawab dan dukung teman, kartu jawaban, indikator kedua, dan tombol Lanjut.

Tutorial halaman ketiga

[Tempel screenshot hasil aplikasi di sini]

Judul Pilih Jawaban Terbaik, kartu dengan garis amber, indikator ketiga, dan tombol Mulai.

Pemeriksaan implementasi

Pemeriksaan build debug melalui task :app:assembleDebug berhasil setelah penyesuaian package dan kode tutorial. Hasil build menunjukkan kode Kotlin dan resource dapat dikompilasi menjadi APK. Aspek yang perlu diperiksa pada aplikasi meliputi perpindahan pilihan chip, susunan lencana, perpindahan halaman tutorial, dan penutupan tutorial melalui tombol.

Kesimpulan

Penerapan modul pertemuan 9 menghasilkan komponen pilihan chip, header profil berbentuk gelombang, susunan lencana tiga kolom, dan tutorial tiga halaman. Struktur adapter dan Fragment mengikuti pola modul. Pengembangan berikutnya dapat menghubungkan pilihan chip dengan filter data pertanyaan.